Roomify

Submission Pack

Technical report, evaluation evidence, architecture, dependencies, repositories and deployment

Team Bootcamp2Roomify

Release v1.0.1 (live, demo mode)
24 October 2026
github.com/Bootcamp2Roomify/roomify_frontend · roomify_backend · roomify_cv

Contents

Technical Report 3
1. Problem 3
2. Approach 3
3. System design 4
4. Models and providers 6
5. Key decisions 6
6. Results 6
7. Limitations 8
8. Future work 8

Evaluation Evidence 9
1. Furniture detection (CV service) 9
2. Redesign plan, visual concept and suggestions (OpenAI) 12
3. Behaviour without an OpenAI key (verified) 14
4. Automated test results (2026-10-26, develop, release v1.0.1) 15

Architecture 16
System overview 16
Request flow 18
Data model 20
Security and privacy 21
Deployment topology (live) 21
Architectural decisions 22

Dependencies 23
Runtimes and container images 23
Backend (roomify_backend/pom.xml) 23
CV service (roomify_cv/requirements.txt, full tree in requirements.lock) 24
Frontend (roomify_frontend/package.json + package-lock.json) 24
External services (not packages) 24

Repositories 25
Licenses 25

Deployment 26
Why this shape 26
Cost 27
Behaviour to expect 27
Steps 27
Alternative: Fly.io (needs a payment method) 28
Risks 28

Technical Report

Redesign more, buy less. Devpost: https://devpost.com/software/roomify-hsc4yn · Code: backend · CV service · frontend

> **Release v1.0.1 (2026-10-26) · live at https://roomify-rooms.vercel.app.** Everything below is implemented, merged, tested and deployed (Vercel + Neon, free plans; Deployment). The OpenAI account was unusable at submission (one key rejected with 401, the other out of credit with 429), so the submitted build runs in **demo mode**: AI features return pre-generated results from a bundled demo library, clearly labelled "Demo" in the app. AI quality, latency, cost and budget-compliance results (§6.2) are therefore **Demo mode (pre-generated results)** — no model numbers are reported or estimated.

1. Problem

People who most want to improve their room — students, renters, young adults on small budgets — are the least served by existing tools (scope):

1. **They can't picture a redesign in their own room.** Inspiration sites show other people's spaces.
2. **They can't see the total cost**, or what to cut when a plan is over budget.
3. **They often can't drill, paint or replace flooring**, yet most ideas assume they can.
4. **Usable furniture gets replaced** instead of kept, moved, repaired or upcycled.

2. Approach

One photo of the real room becomes one organised, saved project:

1. **Detect** the major furniture in the photo (computer vision).
2. **Let the person correct and decide**: Keep, Replace, Remove or Unsure per item; rename, remove or add items the detector got wrong. Detection is treated as a draft, not ground truth.
3. **Collect constraints**: style, colours, room purpose, maximum budget, Rental-Friendly Mode (on by default for renters and dorms), free-text requirements.
4. **Generate a plan with an LLM, then verify it in code**: every kept item is kept, every removed item removed, the costs add up and fit the budget, rental mode contains no permanent work. A plan that breaks a rule is retried once with the reason, otherwise rejected — never shown.
5. **Render a visual concept** by editing the person's own photo with an image model, guided by the verified plan.
6. **Suggest purchases and DIY** within the budget: estimated price, placement, reason, a cheaper alternative, and beginner DIY guides that pass a safety filter.
7. **Save everything** under the person's account; the dashboard reopens any project where it stopped.

The design principle is *the model proposes, the server disposes*: LLM output is untrusted input that must pass deterministic validation before it is stored.

3. System design

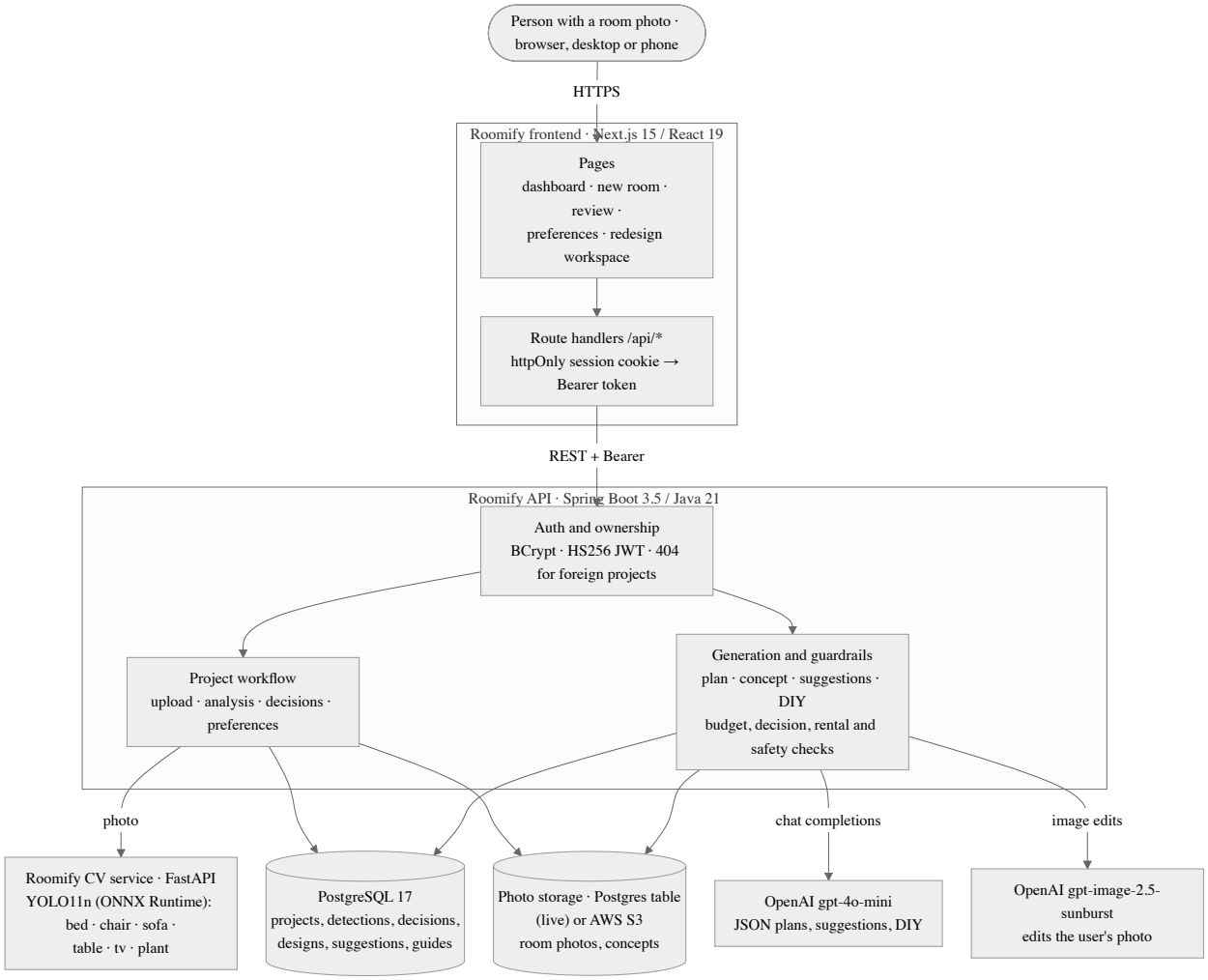


Figure 1. Roomify system architecture.

Service	Stack	Responsibility
Frontend	Next.js 15.5, React 19, TypeScript, Tailwind	Pages; a same-origin bridge (`/api/*` route handlers) that keeps the JWT in an httpOnly cookie and forwards it as a Bearer token
Backend	Spring Boot 3.5, Java 21, PostgreSQL 17 + Flyway	Auth and ownership, workflow state, storage, provider calls, all validation
CV service	FastAPI, Ultralytics YOLO11n	Furniture detection; private, called only by the backend
Storage	AWS S3 (or local disk)	Room photos and generated concepts
AI	OpenAI	Plans, suggestions, DIY (text); concept image (image edits)

3.1 Data flow

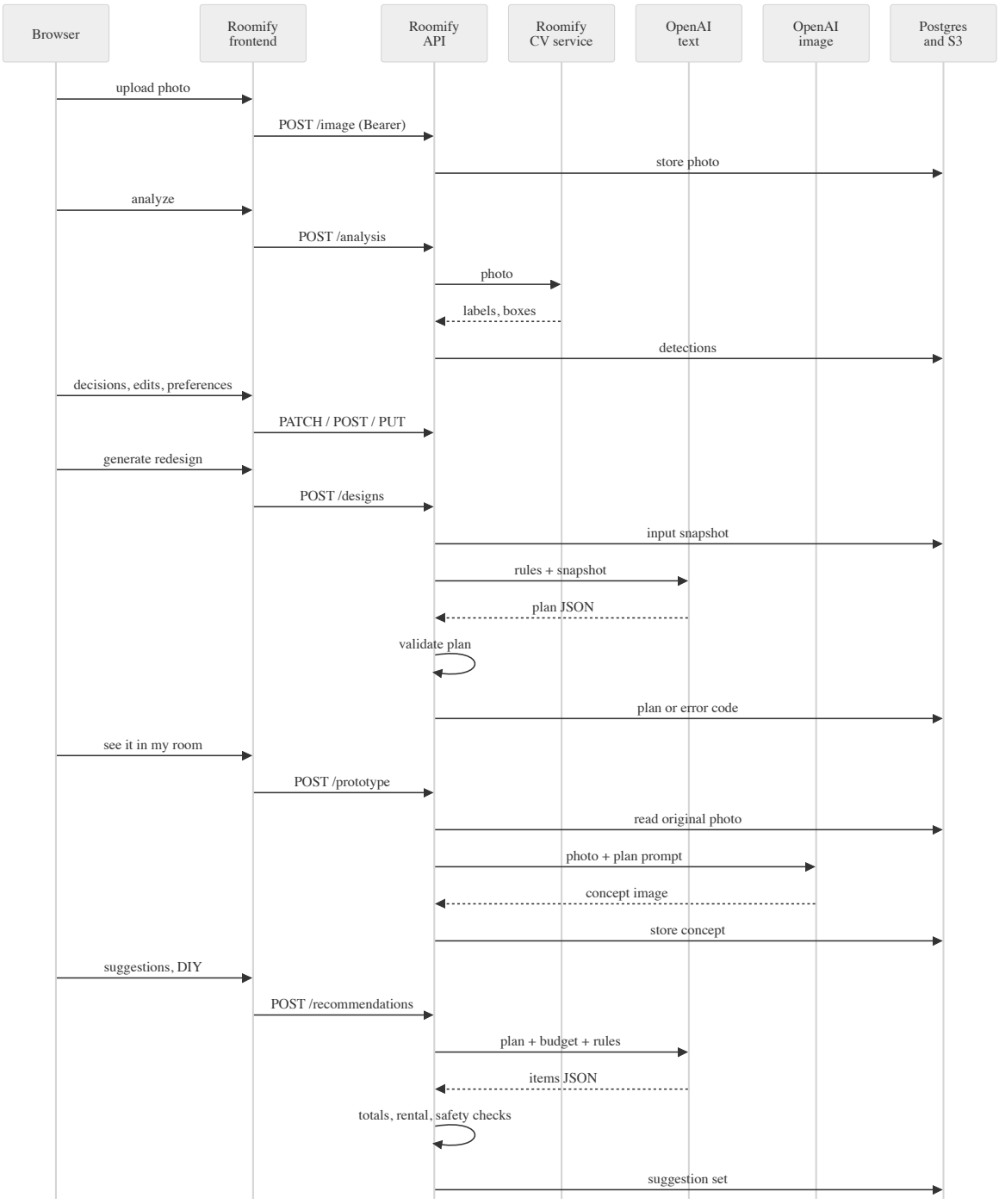


Figure 2. Roomify request flow from upload to plan, visual concept and suggestions.

3.2 Data model (main tables)

`users` → `room_projects` (owner, name, room type, rental status, workflow status) → `room_images` → `detected_objects` (stable UUID, source CV/MANUAL, active flag) → `furniture_decisions`; `project_preferences`; `design_requests` (immutable input snapshot, provider, prompt version) → `design_results` (plan JSON, status, error code) → `design_assets` (concept image); `recommendation_sets` → `recommendation_items` → `recommendation_diy_guides`. Twelve Flyway migrations.

4. Models and providers

Task	Model	Why	Settings
Furniture detection	Ultralytics **YOLO11n**, COCO-pretrained; served live on ONNX Runtime	Runs on CPU in ~32 ms, 5 MB, no training needed; the ONNX export needs no torch and fits a serverless function	confidence ≥ 0.25; six classes kept: bed, chair, sofa, table, tv, plant
Plan, suggestions, DIY	OpenAI **gpt-4o-mini** (Chat Completions, `json_object`)	Cheapest model that follows long JSON schemas	temperature 0.1, ≤ 4000 output tokens, 1 corrective retry
Visual concept	OpenAI **gpt-image-2.5-sunburst** (`images/edits`)	Edits the user's real photo; OpenAI's replacement for the retired `gpt-image-1`, positioned for precise edits (not yet called live)	1024×1024, quality `low`

Model names are configuration. Two provider facts discovered during the build changed the defaults: `gpt-image-1` was shut down by OpenAI on 2026-10-23, and `gpt-4o-mini` shuts down on 2027-01-14 (its replacement `gpt-6-luna` is a reasoning model; the request code now supports it by configuration).

5. Key decisions

1. **Detection is a draft.** COCO has no desk, wardrobe, shelf or lamp class, and measured recall is 0.62 (§6.1). So the review step supports rename, remove and add, low-confidence items are flagged, and analysis failure offers manual entry instead of a dead end.
2. **Validate generated output in code.** Plans must contain every detected item exactly once with its saved decision; instructions must match the decision ("Keep …", "Replace …"); allocations must sum exactly to the total and fit the budget (BigDecimal); rental plans are scanned for permanent work; low-confidence and Unsure items need uncertainty wording. Suggestions get server-computed totals and strictly-cheaper alternatives; DIY guides are Beginner/Easy, cheaper than buying, and rejected if they mention electrical, plumbing, gas or structural work.
3. **Immutable input snapshots.** Each generation stores exactly what the model saw (detections, decisions, preferences, image reference). The concept is rendered from the snapshot's photo, so it always matches its plan, and a design whose inputs changed mid-generation is rejected.
4. **Privacy by construction.** One authorization rule covers every `/api/projects/{id}/**` route; another user's project is indistinguishable from a missing one (404). The browser never holds the JWT. The CV service is designed to stay off the public internet, and its `image_url` input refuses non-public addresses (SSRF).
5. **Estimates, said plainly.** Prices, totals and the concept image are labelled as estimates and concepts everywhere they appear; the plan page states it is not a professional plan.
6. **Budget warnings that stay true.** Stored suggestions always fit the budget at generation time; if the person lowers the budget later, the summary warns and shows what switching to the cheaper options would save.

6. Results

6.1 Detection (measured)

60 COCO val2017 indoor images (held out from training), 159 objects, matched at IoU ≥ 0.5 through the product's HTTP API. Full table, method and failure cases: Evaluation Evidence.

	Precision	Recall	Recall, large objects
All six classes	**0.70**	**0.62**	0.68
Best (bed)	0.86	0.74	0.73
Worst (table)	0.40	0.55	0.67

Latency: 32 ms p50 / 41 ms p95 model time on a laptop CPU.

6.2 Generation

Result	Value
Plan budget compliance	**Demo mode (pre-generated results)**
Plan decision and rental compliance	**Demo mode (pre-generated results)**
Suggestions within budget	**Demo mode (pre-generated results)**
DIY safety-filter rejections	**Demo mode (pre-generated results)**
Latency per AI step	**Demo mode (pre-generated results)**
Cost per redesign	**Demo mode (pre-generated results)**

Method and demo-mode behaviour: Evaluation Evidence §2. Demo results pass the same validators as model output (plan budget and decisions, suggestion budget, DIY safety and rental rules) and are marked `source: "demo"`. Verified with fallback off and no key: every AI action fails with a clear `AI_CONFIGURATION_ERROR` message and leaves the project usable (Evaluation Evidence §3).

6.3 Engineering quality

Suite	Result (develop, 2026-10-26)
Backend JUnit (Postgres 17)	186 passed
CV pytest	22 passed on torch, 22 on ONNX Runtime
Frontend Vitest	235 passed
Playwright, stubbed API, chromium + mobile	39 passed
Playwright, fake Spring API through Next	6 passed
Playwright, **real stack**, demo mode	full journey incl. all AI steps, privacy, logout, CV-down fallback passed
Playwright, **live deployment** (roomify-rooms.vercel.app), demo mode	journey incl. visual concept, privacy, logout passed; journey also passed with a 9.7 MB photo

CI runs on every push and PR in all three repositories. Secret scan of all history (including unreachable objects): no secrets.

7. Limitations

- **Detection** misses about a third of objects, most of them small; it cannot detect desks, wardrobes, shelves, lamps or rugs at all (COCO classes). Measured on 60 COCO images, not on user photos.
- **No spatial understanding.** One photo gives no scale; the plan never claims dimensions or fit.
- **Concepts are not photos.** The image model may change proportions, windows or lighting and does not reproduce kept furniture exactly.
- **Prices are model estimates**, not retailer data; there is no live shopping API.
- **Rental and safety checks are keyword-based** and cannot certify a lease or a DIY step as safe.
- **One room, one photo** per project (as scoped).
- **Cold starts on the free hosting.** The API scales to zero after 5 idle minutes; the next request waits about 10–20 s.
- **Provider dependence.** Two of three models changed status during the build; defaults may need updating again. The text model's deprecation date (2027-01-14) is documented.
- **AI features unverified live.** Plan, concept, suggestion and DIY generation have only run against fake providers; the default image model (`gpt-image-2.5-sunburst`) has never been called.

8. Future work

- Fine-tune a detector on interior datasets with desk, wardrobe, shelf, lamp and rug classes.
- Multiple photos per room for better coverage; optional measurements for fit checks.
- Real product search with live prices and second-hand listings.
- Before/after slider, several concept variations, and a follow-up chat ("make it cosier", "cut cost").
- Run generation asynchronously with progress updates instead of one long request.

Evaluation Evidence

Detection numbers here were measured on 2026-10-24 (tests re-run 2026-10-26) by running the code in this repository; nothing is estimated or copied from model cards. Where something has **not been measured yet**, it says so. Reproduce with the scripts in `evaluation/scripts/`.

1. Furniture detection (CV service)

Set-up. 60 images from **COCO val2017** (held out from YOLO11n's training data, human box annotations): the 60 lowest image ids among the 334 val images that contain a bed or couch, i.e. real indoor rooms. 159 ground-truth objects in Roomify's six classes. Each image was sent through the product path — `POST /v1/analyze` on the running CV service (YOLO11n, `CONFIDENCE_THRESHOLD` 0.25) — and predictions were matched one-to-one per class at **IoU ≥ 0.5**; COCO crowd regions were ignored. "Large" = object covering ≥ 2 % of the image (the furniture a redesign is about).

Class	Ground truth	TP	FP	FN	Precision	Recall	Recall, large objects
bed	34	25	4	9	0.86	0.74	0.73 (33)
sofa	36	20	9	16	0.69	0.56	0.61 (33)
chair	37	22	12	15	0.65	0.59	0.68 (19)
table	11	6	9	5	0.40	0.55	0.67 (9)
tv	15	11	1	4	0.92	0.73	0.80 (10)
plant	26	15	7	11	0.68	0.58	0.67 (9)
all	**159**	**99**	**42**	**60**	**0.70**	**0.62**	**0.68 (113)**

Raw per-image results: `evaluation/data/detection_results.json`; image ids: `evaluation/data/coco_subset_ids.json`.

Latency (same 60 images, Apple-silicon laptop CPU, no GPU): model time p50 **32 ms**, p95 **41 ms**; HTTP round trip p50 35 ms, p95 44 ms, max 278 ms (first request, model warm-up).

What this means for the product. Roughly one object in three is missed and three in ten detections are wrong, so detection is a *starting list*, not an inventory — which is why the review step lets people rename, remove and add items, and why every detection under 60 % confidence is labelled "please check". Beds and TVs are reliable; tables are not (0.40 precision).

Deployed runtime: ONNX Runtime on Vercel (measured 2026-10-26)

The live CV service runs the same YOLO11n weights exported to ONNX on ONNX Runtime (no torch), so it fits a Vercel Python Function. Same 60 images, same script and matching rules:

Runtime	TP	FP	FN	Precision	Recall	Recall, large	Model time p50 / p95
torch (table above), laptop	99	42	60	0.70	0.62	0.68	32 / 41 ms
ONNX Runtime, laptop	101	42	58	0.71	0.64	0.70	38 / 43 ms
ONNX Runtime, live on Vercel	101	42	58	0.71	0.64	0.70	237 / 257 ms

The ONNX path letterboxes to a fixed 640×640 square instead of ultralytics' minimal rectangle, which explains the two extra true positives; this is the same detector, not an improvement. On Vercel (one vCPU) a photo takes about 0.24 s of model time; the HTTP round trip measured from the test laptop in Asia to the US East function was p50 1.4 s, p95 2.0 s, max 3.5 s (cold start included), which says more about the network path than about the service. Memory: about 175 MB for the ONNX service vs about 510 MB with torch. Raw results: `detection_results_onnx_local.json`, `detection_results_vercel.json`.

Failure cases

In each figure, green boxes are ground truth and red boxes are predictions with their confidence. Images are copyright their Flickr authors via COCO (annotations CC BY 4.0) and are shown for evaluation only.

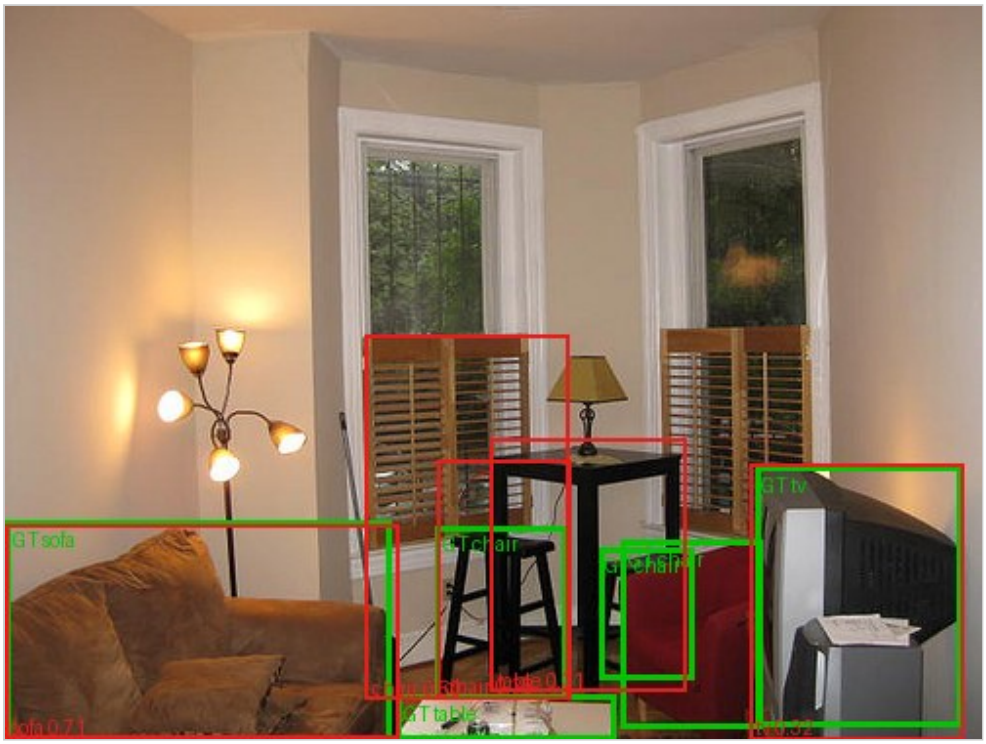


Figure 1. Failure case COCO 47769: a wooden window shutter is detected as a chair (confidence 0.50); two partly hidden chairs and a small table are missed; the floor lamp cannot be detected at all because COCO has no lamp class.



Figure 2. Failure case COCO 39477: the sofa and TV are found, but two hanging planters merge into one box (one false positive and one miss) and three small plants under 3 % of the frame are missed.



Figure 3. Failure case COCO 13923: in a bright living room the sofa (0.90) and large chairs are found, but seven smaller objects (each at most 5.5 % of the image) are missed; there are no false alarms.

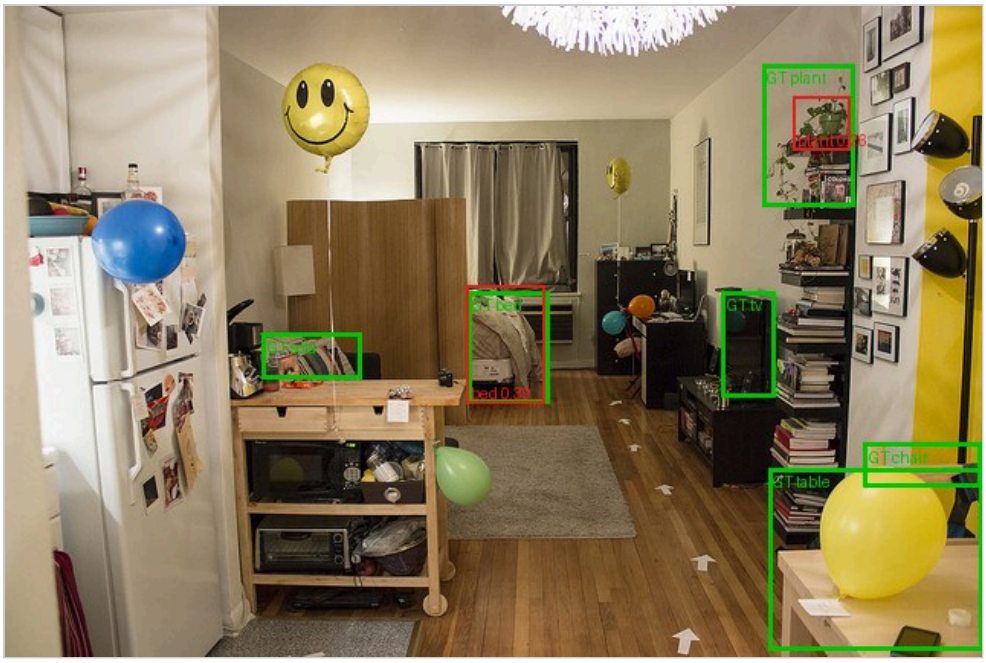


Figure 4. Failure case COCO 14038: in a cluttered studio the bed is found; the TV, a small table and other items under 5 % of the frame are missed. The shelf plant is detected but overlaps its annotation by less than IoU 0.5, so one localisation error counts as both a false alarm and a miss.

Limitations of this measurement

- 60 images is small: one more miss moves a class's recall by ~3 points. Confidence intervals are wide.
- COCO photos are not exactly what users upload (often wider, more cluttered, lower light).
- COCO has **no desk, wardrobe, shelf, lamp or rug classes**, so the most common student-room furniture is never detected; the scope's "storage furniture" relies entirely on manual entry.
- Measured at the service's single confidence threshold (0.25); no threshold tuning was done.

2. Redesign plan, visual concept and suggestions (OpenAI)

Demo mode (pre-generated results). No OpenAI measurements exist. On 2026-10-26 the project's OpenAI account could not serve requests: one key was rejected (`401 invalid_api_key`) and the other had no credit (`429 insufficient_quota`, `credit_balance_exhausted`). The submitted build therefore runs with `AI_DEMO_MODE=true`: the plan, visual concept, suggestions and DIY guides come from a bundled demo library and every such result is marked `source: "demo"` in the API and "Demo" in the UI (see §2.1). None of the values below are estimated or borrowed from model cards; latency and cost of demo results say nothing about the model, so they are not reported.

Result	Value
Plan budget compliance (first attempt / after corrective retry)	**Demo mode (pre-generated results)**
Plan decision compliance (KEEP kept, REMOVE removed)	**Demo mode (pre-generated results)**
Rental-mode compliance (no permanent work)	**Demo mode (pre-generated results)**
Suggestion sets within budget; cheaper-alternative coverage	**Demo mode (pre-generated results)**
DIY guides rejected by the safety filter	**Demo mode (pre-generated results)**
Visual-concept quality (architecture preserved, kept items visible)	**Demo mode (pre-generated results)**
Latency p50 / p95: plan, concept, suggestions, DIY	**Demo mode (pre-generated results)**
Cost per full redesign (tokens × list price on the run date)	**Demo mode (pre-generated results)**

2.1 What demo mode does (verified by tests)

- Four cases chosen by room purpose: living room (`LIVING`), bedroom (`SLEEP`), home office (`STUDY`), student room (`STUDY_AND_SLEEP`, `MULTIPURPOSE`, default).
- The demo plan is built from the request — every detected object with its KEEP / REPLACE / REMOVE / UNSURE decision, the budget, currency and rental flag — and must pass the same `DesignPlanValidator` as model output. Suggestions and DIY guides pass the same contract, safety and rental checks.
- The concept image is a CC0 stock photo for the room type (credits: `src/main/resources/demo/CREDITS.md`), **not** an edit of the user's photo; the UI says so.
- With a funded key (`AI_DEMO_MODE=false`) the app calls OpenAI; `AI_DEMO_FALLBACK=true` (default) answers from the library when a call fails (401, 429, timeout, invalid output) and logs `AI demo result used: feature=… case=… reason=…`.
- Tests: `DemoLibraryTest` (every case through the validators, odd and zero budgets, TWD, rental, low-confidence items) and `DemoFallbackTest` (each endpoint in demo mode without calling the provider; quota, rejected-key, timeout, unexpected-error and invalid-output fallback).

The live-AI measurement harness is ready for when a funded key is available:

- the backend logs one content-free line per provider call (`AI usage kind=text|image model=… promptTokens=… completionTokens=… reasoningTokens=… latencyMs=…`);
- `tests/live` in roomify_frontend runs the full journey with `LIVE_AI=1`.

Planned measurements (same rooms, fixed budgets of 150 / 300 / 600 USD, rental on and off):

Metric	How it will be measured
Plan budget compliance	share of first-attempt plans whose allocation sum equals the total and is ≤ budget (before the server's validator); share stored after the one corrective retry
Decision compliance	share of plans where every KEEP item is kept and every REMOVE item removed (validator outcome)
Rental compliance	share of rental-mode plans rejected for permanent-modification wording
Suggestion budget fit	share of first-attempt suggestion sets within budget; cheaper-alternative coverage
DIY safety	share of guides rejected by the safety filter
Latency	p50 / p95 per call type from the usage log
Cost	tokens × OpenAI list prices on the day of the run

3. Behaviour without an OpenAI key (verified)

`MissingAiKeyTest` runs the real OpenAI provider beans with no key; a blank key, an unedited `.env.example` placeholder and a key rejected by OpenAI (401/403) all count as "not configured".

Action	Response	State afterwards
Generate plan (twice)	`503 AI_CONFIGURATION_ERROR`	attempt stored as FAILED with that code, nothing pending, project status unchanged, retry allowed
Visual concept	`503 AI_CONFIGURATION_ERROR`	no image asset written
Suggestions	`503 AI_CONFIGURATION_ERROR`	no suggestion set written
DIY guide	`503 AI_CONFIGURATION_ERROR`	no guide written; saved plan still readable

The UI shows: "AI generation is not configured on this server yet (no valid OpenAI key). Your photo, furniture and preferences are saved — try again once AI is enabled." (frontend tests cover all four actions; the real-stack browser journey shows it for the plan).

4. Automated test results (2026-10-26, develop, release v1.0.1)

Suite	Result
Backend (JUnit, Postgres 17)	186 passed (25 for demo mode)
CV service (pytest)	22 passed on torch, 22 passed on ONNX Runtime
Frontend unit (Vitest)	235 passed
Browser, stubbed API (Playwright, chromium + mobile)	39 passed
Browser, fake Spring API through Next (Playwright)	6 passed
Browser, real stack, `AI_DEMO_MODE=true` (Playwright `tests/live`, `LIVE_AI=1 LIVE_DEMO=1`)	full journey incl. plan, concept, suggestions, DIY and over-budget warning; privacy; logout; CV-down manual entry: all passed
Browser, **live deployment** https://roomify-rooms.vercel.app, demo mode (same suite, `LIVE_BASE_URL`)	journey incl. plan, visual concept (image loaded), suggestions and DIY; privacy; logout: passed. CV-down skipped (needs the CV service stopped). The journey also passed with a 9.7 MB, 6000×4000 photo (shrunk in the browser below Vercel's 4.5 MB body limit). One earlier run timed out at registration (15 s limit) while a new API instance cold-started; it passed on rerun

Architecture

Roomify turns one photo of a person's own room into a saved, budget-aware and rental-aware redesign project. It is built as three services — a web frontend, an API and a computer-vision service — plus PostgreSQL, object storage and the OpenAI API. This document describes how they fit together, how a request flows through them, what is stored, and how privacy is enforced.

System overview

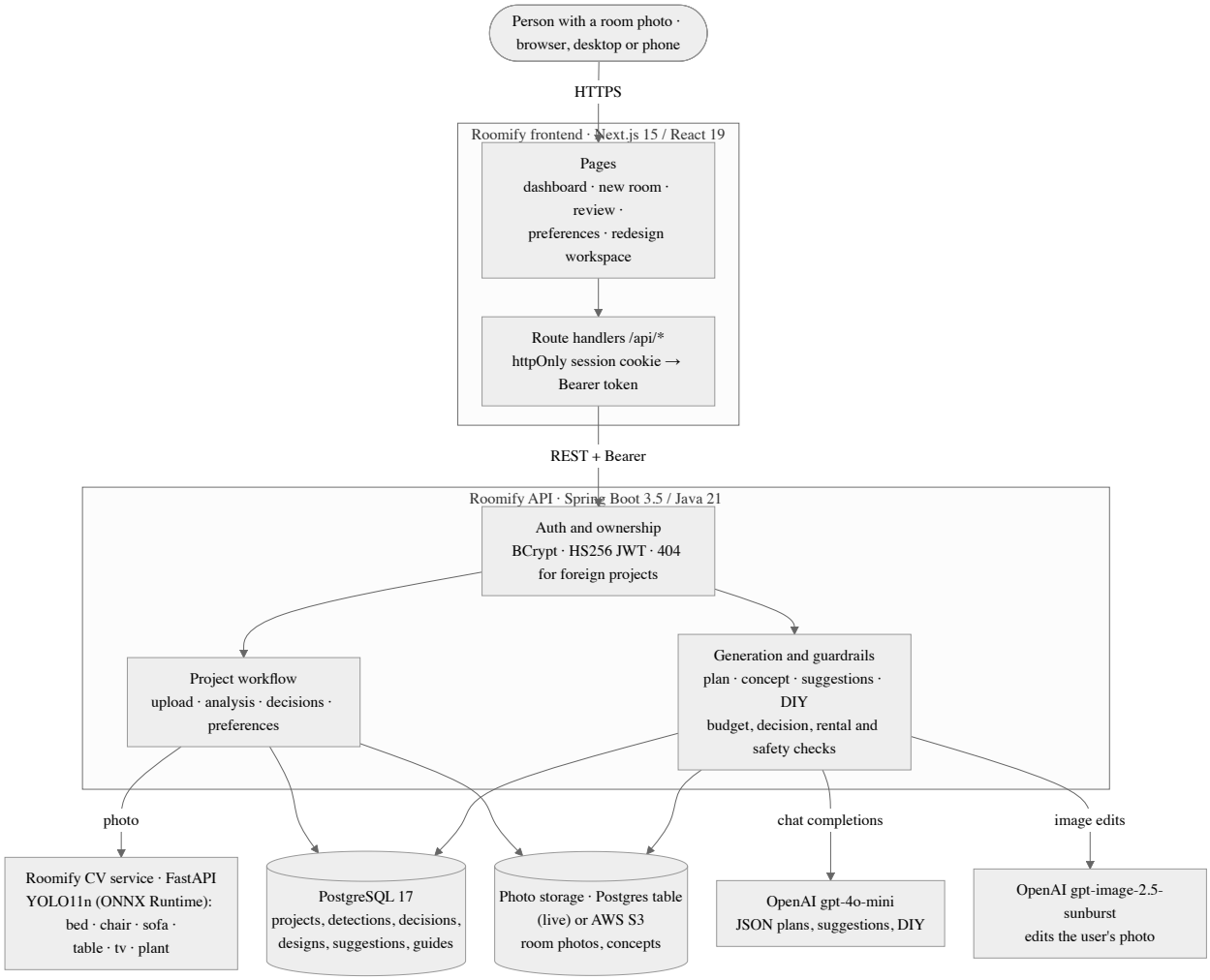


Figure 1. Roomify system architecture: browser, Roomify frontend, Roomify API, Roomify CV service, OpenAI, PostgreSQL and object storage.

Component	Technology	Responsibility
Roomify frontend	Next.js 15.5, React 19, TypeScript, Tailwind CSS	Pages for the whole journey; same-origin route handlers (`/api/*`) that keep the access token in an httpOnly cookie and forward it to the API as a Bearer token
Roomify API	Spring Boot 3.5, Java 21	Accounts and ownership, project workflow, image storage, calls to the CV service and OpenAI, and all validation of generated output
Roomify CV service	FastAPI, Ultralytics YOLO11n	Detects six furniture classes in a photo; reachable only by the API
PostgreSQL 17	Flyway migrations V1–V12	Users, projects, detections, decisions, preferences, design history, suggestions, DIY guides
Object storage	AWS S3 (local disk in development)	Original room photos and generated concept images
OpenAI API	Chat Completions (`gpt-4o-mini`), Images edits (`gpt-image-2.5-sunburst`)	Redesign plans, suggestions and DIY guides as JSON; the visual concept

The browser never calls the API or the CV service directly and never sees the access token. All three services ship as pinned container images.

Request flow

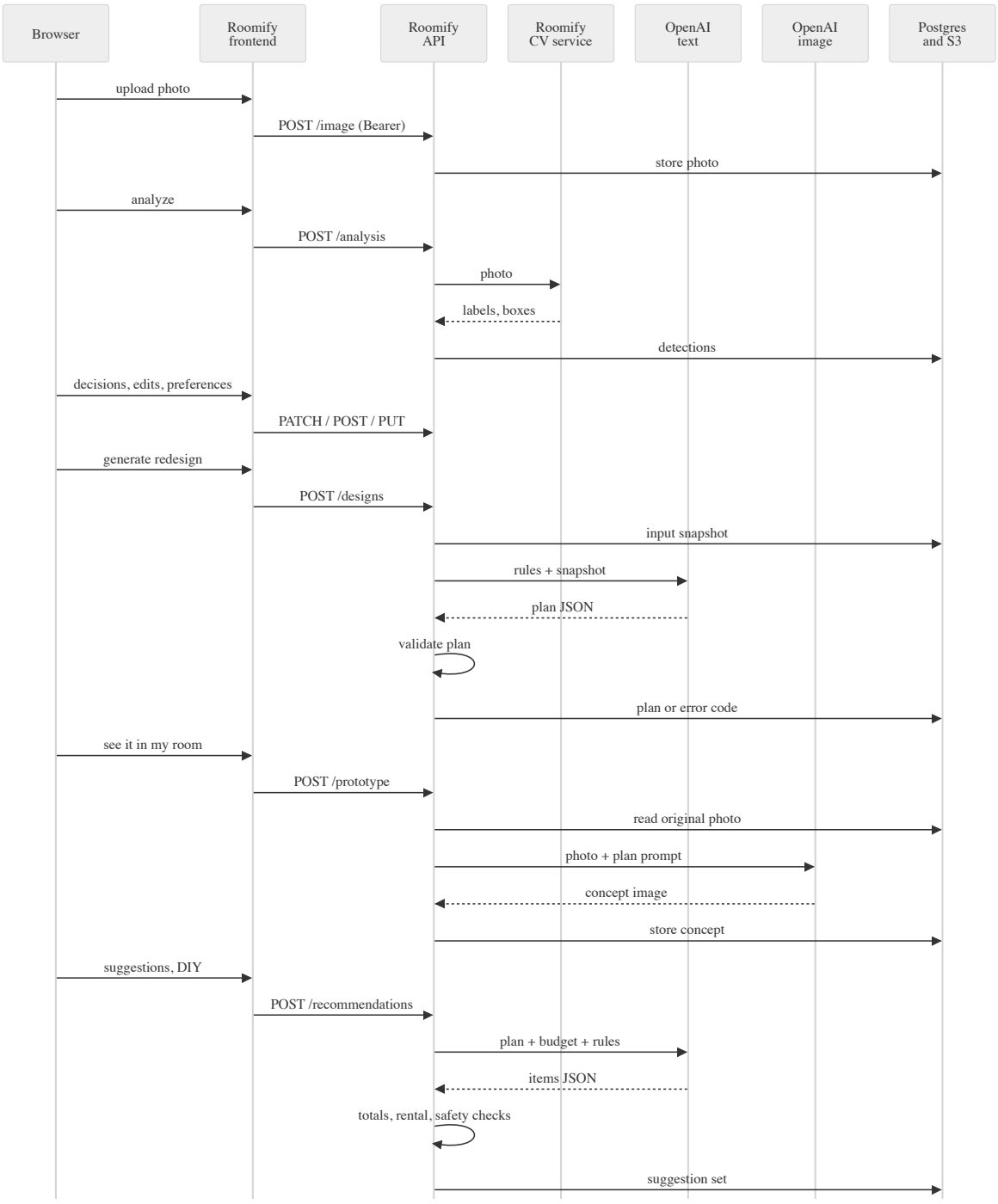


Figure 2. Roomify request flow from photo upload to plan, visual concept, suggestions and DIY guide.

1. **Upload.** The photo is validated (JPG or PNG, size and signature), stored, and linked to the project.
2. **Analysis.** The API sends the photo to the CV service and stores each detection with a stable UUID.
3. **Review.** The person sets Keep, Replace, Remove or Unsure per item and can rename, remove or add items; added and renamed items are marked as user-confirmed.
4. **Preferences.** Style, colours, purpose, budget, Rental-Friendly Mode and free-text requirements.

5. **Plan.** The API stores an immutable input snapshot, asks the text model for a plan in a fixed JSON schema, then validates it in code: every item with its saved decision, matching instructions, exact budget arithmetic within the budget, no permanent work in rental mode, uncertainty wording for low-confidence items. One corrective retry; otherwise the attempt is stored as failed with a code.
6. **Visual concept.** The photo recorded in the snapshot is edited by the image model with a prompt derived from the validated plan, then stored.
7. **Suggestions and DIY.** The text model proposes items; the API rounds prices, computes totals, checks cheaper alternatives and rental wording, and stores the set. DIY guides must be beginner level, cost no more than buying, and pass a safety filter.

No database transaction stays open while a provider is working, and nothing partially generated is ever shown.

Data model

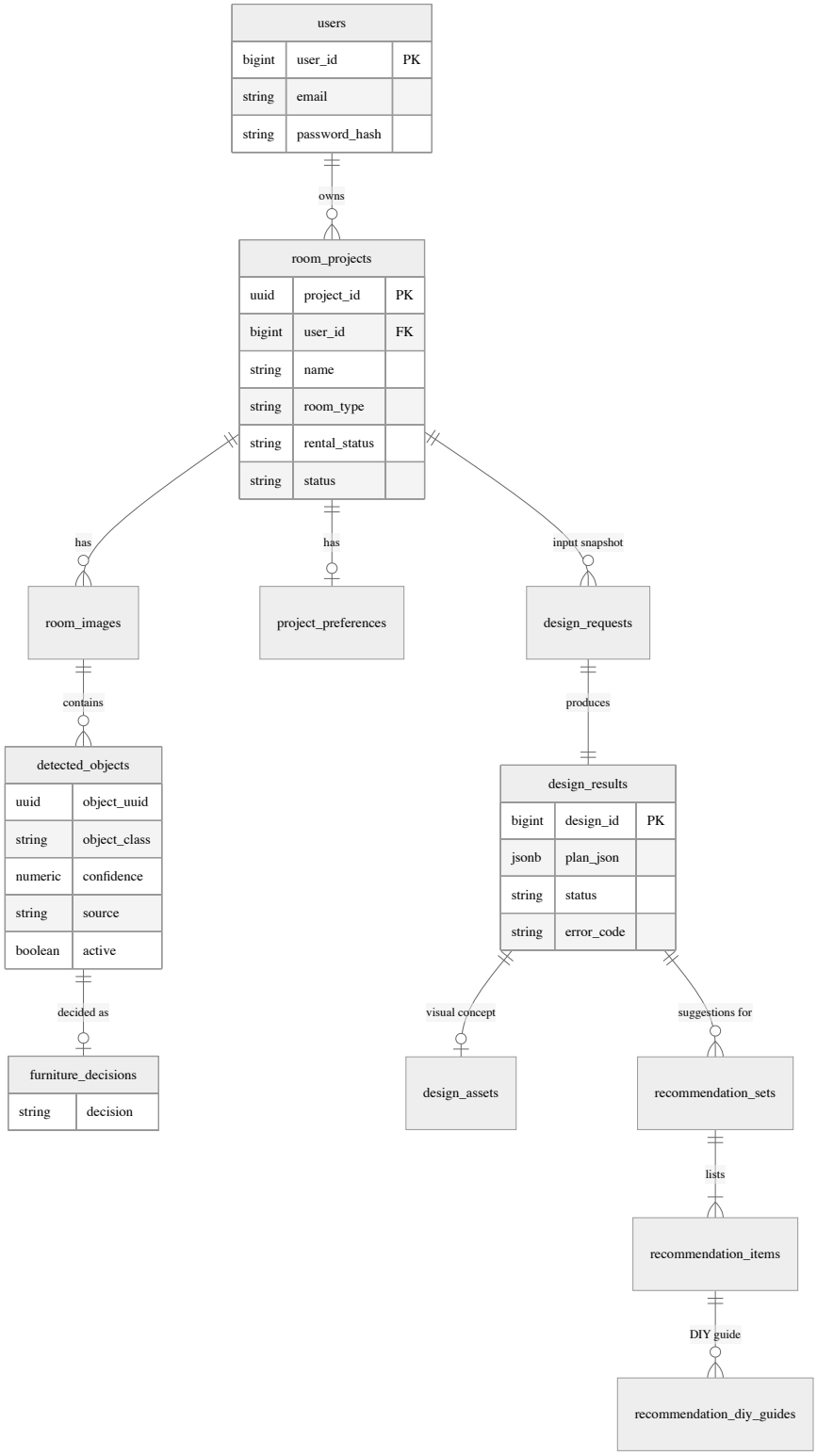


Figure 3. Roomify main tables and relationships.

Each generation keeps the exact input it was made from (`design_requests.input_snapshot`), the provider, model and prompt version, the outcome and a stable error code. Detections are never deleted, only deactivated, so design history stays explainable. Money is stored as `NUMERIC(12,2)` and handled as

`BigDecimal`; database CHECK constraints repeat the most important rules (totals, statuses, currency).

Security and privacy

Concern	Design
Authentication	Email and password (BCrypt); stateless HS256 access tokens, 1 hour by default
Token handling	Stored only in an httpOnly, SameSite=Lax cookie (Secure in production) set by the frontend's route handlers
Authorisation	One rule covers every `/api/projects/{id}/**` route; another user's project returns the same 404 as a missing one
Generated content	Treated as untrusted input; validated before storage; provider error messages never shown or logged
CV service	Detection endpoints require a bearer token known only to the backend (it is publicly reachable on Vercel); its `image_url` input rejects non-public targets (SSRF) and does not follow redirects
Secrets	Only `.env.example` files in git; full history scanned with no findings

Deployment topology (live)

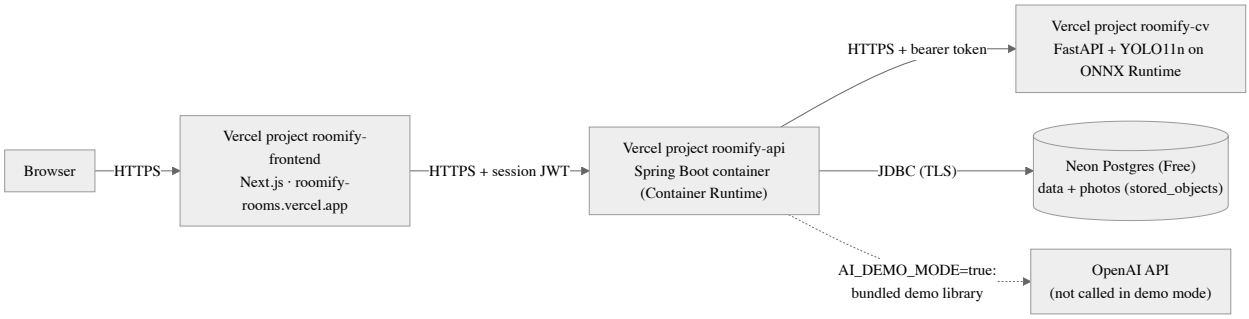


Figure 4. Roomify live deployment: three Vercel projects (Next.js frontend, Spring Boot container, FastAPI CV function) and Neon Postgres.

Live at https://roomify-rooms.vercel.app on free plans with no payment card: the Next.js frontend, the Spring Boot API (Vercel Container Runtime) and the CV service (Vercel Python Function, YOLO11n on ONNX Runtime) are three Vercel projects; Postgres on Neon holds the data and the photos. Reasons, cold-start behaviour, steps and the Fly.io alternative are in the Deployment document.

Architectural decisions

Decision	Reason
Three services instead of one	Keeps the vision runtime out of the JVM host and lets each part scale, deploy or be replaced independently (the CV service moved from torch to ONNX Runtime for hosting without touching the backend)
Backend-for-frontend bridge in Next.js	The token never reaches browser JavaScript; no CORS surface on the API
"The model proposes, the server disposes"	LLM output is checked by deterministic code before it is stored or shown
Immutable input snapshots	The concept always matches its plan; changed inputs mid-generation are detected
Model names are configuration	OpenAI retired one model during the build; switching needs no code change

Dependencies

Every version below is pinned exactly in the file named in the last column. Resolved on 2026-10-24 from the `develop` branches; nothing uses ranges (`^`, `~`, `latest`, bare major tags).

Runtimes and container images

Component	Version	Pinned in
Java (Temurin)	21	`roomify_backend/pom.xml` (`java.version`), images below
Maven	3.9.16	`roomify_backend/.mvn/wrapper/maven-wrapper.properties`
Backend build image	`maven:3.9.16-eclipse-temurin-21`	`roomify_backend/Dockerfile`, `compose.yaml`
Backend runtime image	`eclipse-temurin:21.0.12_8-jre`	`roomify_backend/Dockerfile`
PostgreSQL	`postgres:17.11`	`roomify_backend/compose.yaml`, CI service
Python	3.14 (`python:3.14.9-slim`)	`roomify_cv/Dockerfile`, CI
Node.js	26.4.0	`roomify_frontend/.nvmrc`, `package.json` `engines`, CI
GitHub Actions	checkout v7.0.1, setup-java v6.1.0, setup-node v7.1.0, setup-python v7.1.0	each repo's `.github/workflows/ci.yml`, pinned by commit SHA

Backend (`roomify_backend/pom.xml`)

Versions not written in the POM come from the Spring Boot 3.5.0 parent BOM, which fixes them exactly.

Dependency	Version	Use
spring-boot-starter-web / validation / data-jpa	3.5.0	REST API, bean validation, JPA
spring-boot-starter-security / oauth2-resource-server	3.5.0 (Spring Security 6.5.0)	BCrypt, HS256 JWT issuing and verification
flyway-core / flyway-database-postgresql	11.7.2	Schema migrations V1–V12
org.postgresql:postgresql	42.7.5	JDBC driver
software.amazon.awssdk s3 / signin	2.55.5 (`aws.sdk.version`)	Image storage on S3
test: spring-boot-starter-test 3.5.0, spring-security-test 6.5.0, networknt json-schema-validator 1.5.9		Unit/integration tests, contract schema checks

CV service (`roomify_cv/requirements.txt`, full tree in `requirements.lock`)

Dependency	Version	Use
fastapi	0.142.2	HTTP API
uvicorn[standard]	0.54.0	ASGI server
python-multipart	0.0.32	File uploads
ultralytics	8.4.171	YOLO11 inference
torch / torchvision (via ultralytics)	2.14.1 / 0.29.1	CPU inference (CPU wheels in Docker/CI)
pillow	12.3.0	Image decoding
httpx	0.28.1	`image_url` fetching
dev: pytest	9.1.1	Tests (`requirements-dev.txt`)
Model weights	`yolo11n.pt` (Ultralytics release asset)	Baked into the image at build time

Frontend (`roomify_frontend/package.json` + `package-lock.json`)

Dependency	Version	Use
next	15.5.27	App Router, route handlers, middleware
react / react-dom	19.0.0	UI
gsap / @gsap/react	3.13.0 / 2.1.2	Landing-page motion
dev: typescript 5.6.2, tailwindcss 3.4.13, eslint 8.57.1 + eslint-config-next 15.5.27, vitest 5.0.3, @testing-library/react 16.2.0, @playwright/test 1.63.0		Build, lint, tests

External services (not packages)

Service	Model / API	Configured by
OpenAI Chat Completions	`gpt-4o-mini`, `response_format: json_object`	`AI_MODEL`, `OPENAI_API_KEY`
OpenAI Images (edits)	`gpt-image-2.5-sunburst`, 1024×1024, `low` quality (replaces `gpt-image-1`, retired 2026-10-23; not yet called live)	`AI_IMAGE_MODEL`, `AI_IMAGE_SIZE`, `AI_IMAGE_QUALITY`
AWS S3	any bucket in `AWS_REGION`	`STORAGE_TYPE=s3`, `S3_BUCKET`

Model names are configuration, not code: switching models needs no rebuild.

Repositories

Repo	Visibility now	Recommendation	README
roomify_frontend	Public	**Keep public**	setup, pages, env var, all test suites
roomify_backend	Public	**Keep public**	setup, endpoint map, every env var, tests
roomify_cv	Public	**Keep public**	model, API, private-network note, venv + container

Why public: judges can read and run the code without access requests; the full history of all three repos (all refs plus unreachable objects) was scanned for AWS keys, OpenAI/Google keys, private keys and credential-like assignments on 2026-10-24 and **nothing was found**; only `.env.example` files with placeholders have ever been committed, and `.env` is git-ignored everywhere.

No visibility change is needed. (None will be made without the project owner's approval.)

Licenses

Repo	License
roomify_frontend	MIT
roomify_backend	MIT
roomify_cv	**AGPL-3.0** (relicensed 2026-10-24 to match Ultralytics YOLO; earlier MIT notice kept in `NOTICE`)

The backend calls the CV service over HTTP as a separate program; the frontend never touches it.

Deployment

Status (2026-10-26): live at https://roomify-rooms.vercel.app, in demo mode (`AI_DEMO_MODE=true`). Everything runs on free plans with no payment card: three Vercel Hobby projects and a Neon Free database.

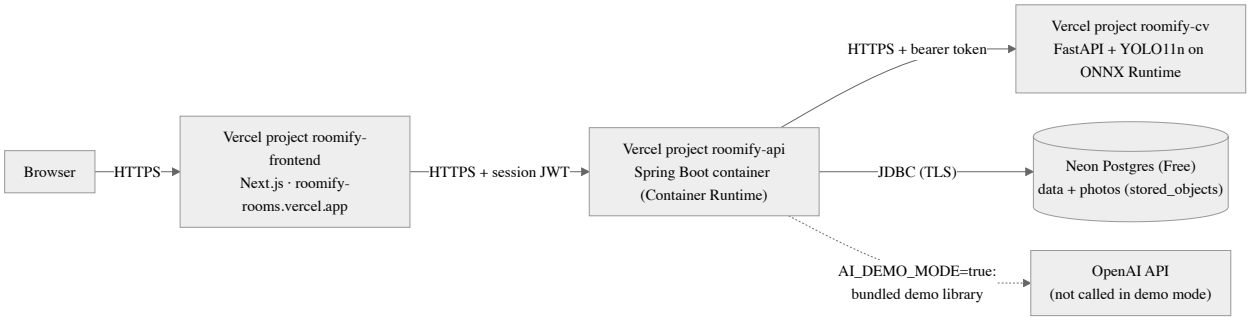


Figure 1. Roomify live deployment: three Vercel projects (Next.js frontend, Spring Boot container, FastAPI CV function) and Neon Postgres.

Piece	Where	URL
Frontend (Next.js)	Vercel project `roomify-frontend`	https://roomify-rooms.vercel.app
Backend (Spring Boot, Java 21)	Vercel project `roomify-api`, Container Runtime (beta)	https://roomify-api.vercel.app/api/health
CV service (FastAPI, YOLO11n)	Vercel project `roomify-cv`, Python Function on ONNX Runtime	https://roomify-cv.vercel.app/health
Database and photos	Neon Postgres Free, `us-east-2`	–

Why this shape

The owner asked for the backend on Vercel too, and for a host with no payment card. What was checked:

- **Vercel runs the JVM now.** Vercel Functions accept an OCI image built from a Dockerfile (Container Runtime, beta on all plans, Hobby included). It is not a long-running server: an instance serves requests, scales to zero after 5 idle minutes and must open its port within 28 s of a cold start. The backend fits that model: it keeps no state in memory, has no background jobs or schedulers (every request is handled synchronously), and stores photos in Postgres.
- **The CV service could not ship with torch.** The torch build uses about 510 MB of memory and far more than Vercel's 500 MB Python bundle limit. It now also runs on ONNX Runtime (roomify_cv#15): about 175 MB, same detections in practice (Evaluation Evidence §1).
- **Rejected:** Hugging Face Docker Spaces (a PRO subscription is required since 2026; creating the Space returned 402), Koyeb (card required since February 2026, per third-party reports), Render Free (512 MB per service, too small for the JVM plus YOLO in one container, and a split would expose the CV service on a second public URL anyway), Fly.io (payment method required).
- **No S3 or AWS key.** `STORAGE_TYPE=database` stores photos in a `stored_objects` table (migration V12). Vercel Blob has no Java SDK, and the database is already there. Neon Free holds 0.5 GB, plenty for judging; a demo-mode redesign adds about 60 kB.

- **The CV service is public, so it checks a token.** Vercel has no private network on Hobby. The backend sends `Authorization: Bearer $VISION_SERVICE_API_TOKEN`; the CV service rejects any other call to `/v1/analyze` or `/detect` with 401 (`CV_API_TOKEN`). `/health` stays open.

Cost

Piece	Plan	Monthly
Frontend, backend, CV service	Vercel Hobby (personal, non-commercial: fine for judging)	$0 within Hobby's included usage
Database and photos	Neon Free (0.5 GB, scales to zero after 5 min idle)	$0
OpenAI	not called while `AI_DEMO_MODE=true`	$0

Behaviour to expect

- **Cold starts.** After 5 idle minutes the backend scales to zero. The next request waits for the container: Spring starts in about 8 s on Vercel thanks to a class-data-sharing archive built into the image (13–18 s without it), and the very first request after a deploy took 22 s including the image pull. Neon adds up to a few seconds if the database was also asleep. Warm requests take under 1 s. The CV function cold-starts in about 1 s and analyses a photo in about 0.24 s. A scheduled GitHub Actions workflow (`.github/workflows/keep-warm.yml`, every 5 minutes) pings both services to keep them warm during judging; GitHub may delay scheduled runs, so a visitor can still hit a cold start, and Vercel may start a second instance (with its own cold start) when requests overlap.
- **Upload size.** Vercel Functions reject request bodies over 4.5 MB. The frontend re-encodes photos above 4 MB as JPEG (longest side 2560 px) before uploading (roomify_frontend#38).
- **Demo mode.** The plan, visual concept, suggestions and DIY guides come from the bundled demo library (fitted to the user's furniture decisions and budget; the concept is a stock photo for the room type), and every result is marked `source: "demo"` and labelled in the app. With `AI_DEMO_MODE=false` and a funded `OPENAI_API_KEY` the app calls OpenAI and falls back to the library if a call fails (`AI_DEMO_FALLBACK=true`). Real AI calls can take up to ~2 minutes; Hobby functions allow 300 s.

Steps

Prerequisites: a Vercel account (`vercel login`) and a Neon database. All three projects live in the same Vercel team.

1. **Database** — create a Neon project (Free) and note the host, database, user and password. The backend runs the Flyway migrations on first start; on an empty database that first start can miss Vercel's 28 s port deadline once (one 500), after which it is migrated and starts in about 8 s.
2. **CV service**

```
cd roomify_cv
deploy/vercel/build.sh /tmp/roomify-cv-vercel     # exports yolo11n.onnx, assembles
the bundle
cd /tmp/roomify-cv-vercel
vercel project add roomify-cv && vercel link --yes --project roomify-cv
printf %s yolo11n.onnx | vercel env add MODEL_PATH production
openssl rand -hex 32 | tr -d '\n' | vercel env add CV_API_TOKEN production   # keep
a copy for step 3
vercel deploy --prod
```

3. **Backend** (from the repo root; `.vercelignore` uploads only `Dockerfile.vercel`, `pom.xml` and `src`)

```
cd roomify_backend
vercel project add roomify-api && vercel link --yes --project roomify-api
vercel project update --framework container --yes   # pom.xml would otherwise hide
the Dockerfile
# production environment variables (vercel env add NAME production):
#   SPRING_DATASOURCE_URL=jdbc:postgresql://<neon-host>/<db>?sslmode=require
#   DB_USERNAME, DB_PASSWORD, JWT_SECRET=$(openssl rand -hex 32)
#   VISION_SERVICE_BASE_URL=https://roomify-cv.vercel.app
#   VISION_SERVICE_API_TOKEN=<CV_API_TOKEN from step 2>,
VISION_SERVICE_READ_TIMEOUT=30s
#   AI_DEMO_MODE=true, STORAGE_TYPE=database, PORT=8080,
SPRING_DATASOURCE_HIKARI_MAXIMUM_POOL_SIZE=3
vercel deploy --prod
curl https://roomify-api.vercel.app/api/health
```

   Neon requires TLS, so the JDBC URL is set whole with `sslmode=require` (it overrides `DB_HOST` / `DB_PORT` / `DB_NAME`). A small pool per instance keeps Neon's connection count low when Vercel runs several instances.
4. **Frontend** — project `roomify-frontend` (Next.js preset, Node 24.x), environment variable `ROOMIFY_API_URL=https://roomify-api.vercel.app`, then `vercel deploy --prod`. The public alias `roomify-rooms.vercel.app` was added as a project domain (`vercel domains add`) so it is not behind Vercel's deployment protection.
5. **Smoke test** — the live browser suite against the deployment, including a visual concept: `LIVE_BASE_URL=https://roomify-rooms.vercel.app LIVE_AI=1 LIVE_DEMO=1 npx playwright test -c playwright.live.config.ts`.
6. **After judging** — delete or pause the three Vercel projects; delete the Neon project.

Alternative: Fly.io (needs a payment method)

`roomify_backend/fly.toml` and `roomify_cv/fly.toml` remain for a host with always-on 1 GB machines, a private network for the CV service and S3 storage (`STORAGE_TYPE=s3` with a bucket-scoped IAM key). That setup costs about $13.40/month (two shared-cpu-1x machines at $6.70) and avoids cold starts.

Risks

- Container Runtime is a Vercel **beta**. If it changes, the Fly.io configuration is the fallback.
- `gpt-4o-mini` (default text model) is shut down by OpenAI on **2027-01-14**; switch to `gpt-6-luna` with `AI_TEMPERATURE=` and `AI_REASONING_EFFORT=low` before then (config only).

- Vercel Hobby is for non-commercial use; move to Pro ($20/month) for anything beyond judging.
- Before turning demo mode off on a public URL, set a monthly usage limit in the OpenAI dashboard.